

Master Degree Computer Engineering
Computer Architecture

String Comparator

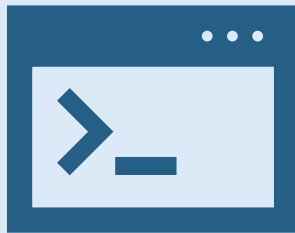
GPU Version

Megan Maremmani
Eleonora Sgorbini
Sebastiano Pala

ALGORITHM: The idea



The purpose of the algorithm is to **count how many times a target word appears in a text file.**



EXAMPLE:

*This is my dog. His name is **Max**. **Max** is a good dog. **Max** is very happy. He likes to run. He runs in the park. He runs in the garden. He runs every day. **Max** likes to play with his ball. The ball is red. It is a big red ball. **Max** runs to the red ball. He plays with the ball in the morning. He plays with the ball in the afternoon.*

EXPECTED OUTPUT:

“Max” occurrences: 5



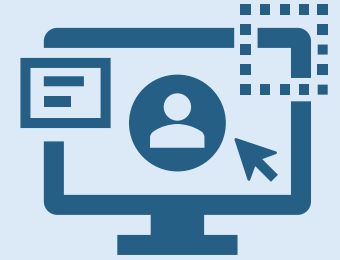
GOALS



The primary objective is to calculate the correct number of instances of a specified string while maximizing throughput

USER POV

Throughput should be at least 5 GBytes/s for large files (over 4 GBytes)



DEVELOPER POV



Develop a GPU version that outperforms the CPU version



TOOLS:



**NVIDIA Nsight
compute**



**Visual Studio
code**



**Chrono libraries
for timing**

```
chrono::steady_clock::time_point start = chrono::steady_clock::now();  
// algorithm execution  
chrono::steady_clock::time_point end = chrono::steady_clock::now();  
  
// compute overall duration  
chrono::milliseconds duration = chrono::duration_cast<chrono::milliseconds>(end - start);
```



NVIDIA GeForce RTX 5060 MOBILE



Threads per warp: 32

Stream Multiprocessors (SM): 26

Max warps per SM: 48

Max thread blocks per SM: 24

Max number of threads for SM: 1536

Max thread block size: 1024

Registers per SM: 65536

Memory details

VRAM:

8 Gbytes

L1 cache for SM:

128 KB (per SM)

L2 cache:

32 MB

Max shared memory for SM:

102400 Bytes (100 KBytes)

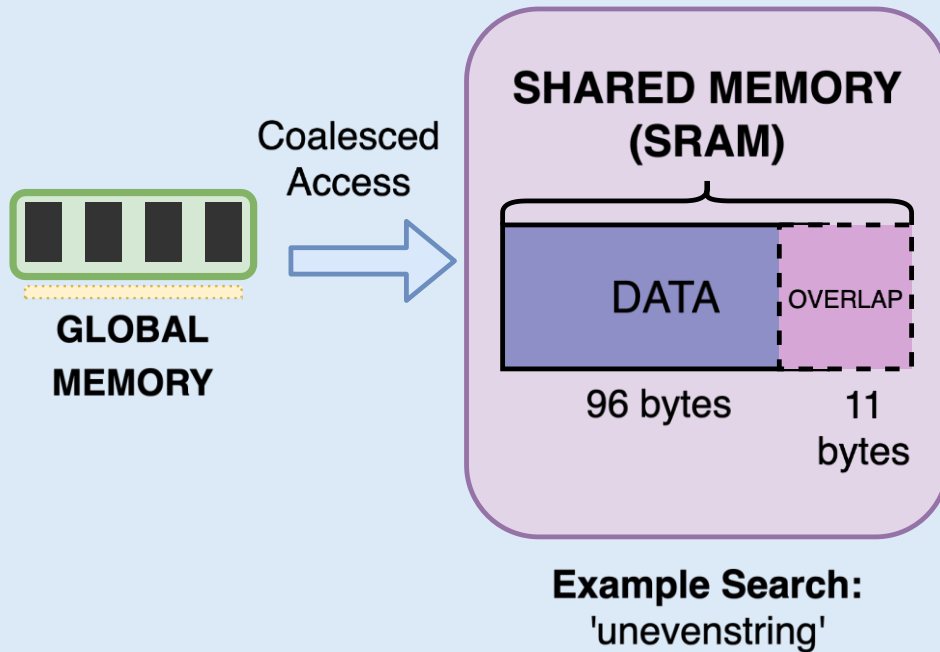


nVIDIA®

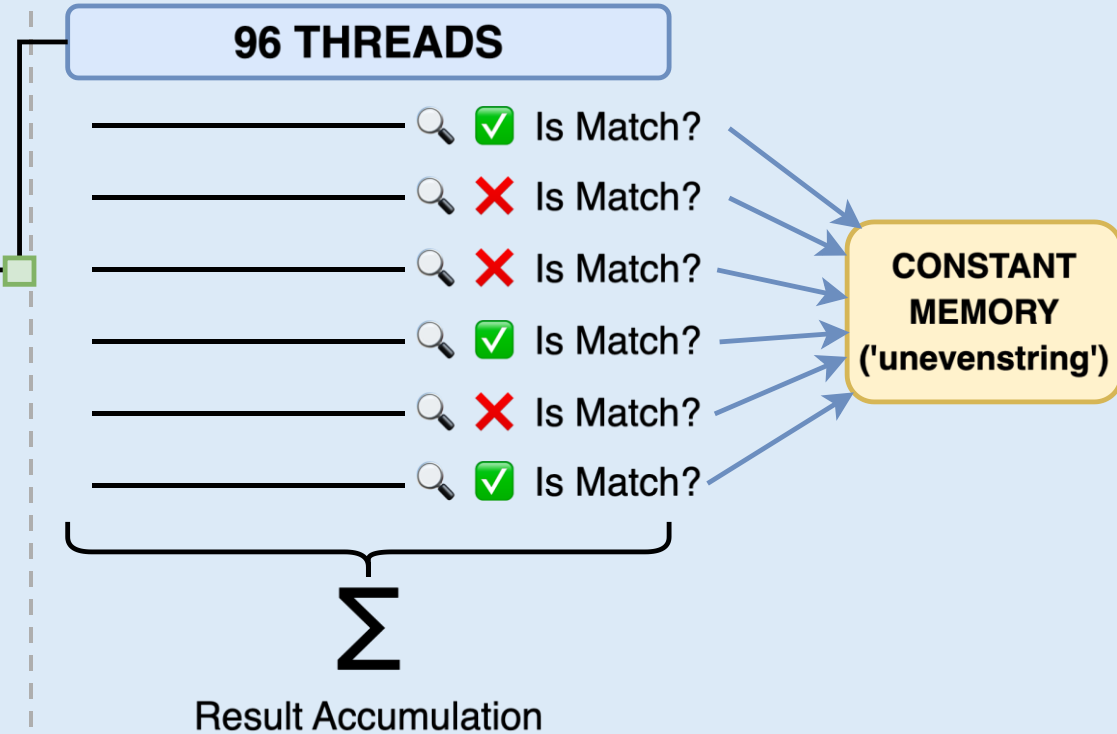
STRIDE CODE: Inner Working



FETCH PHASE



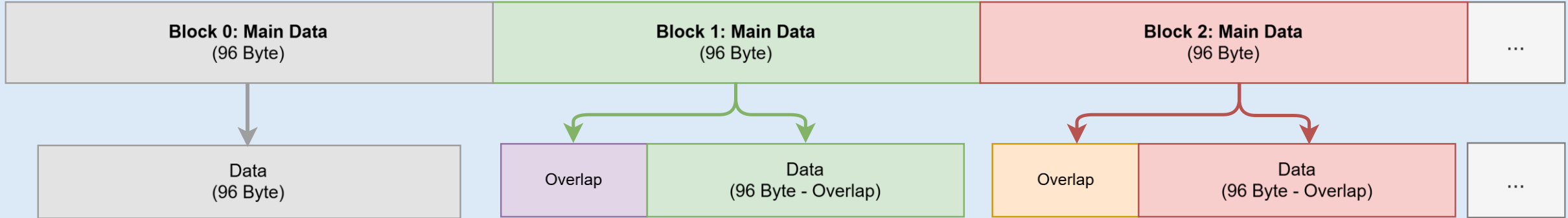
MATCH PHASE



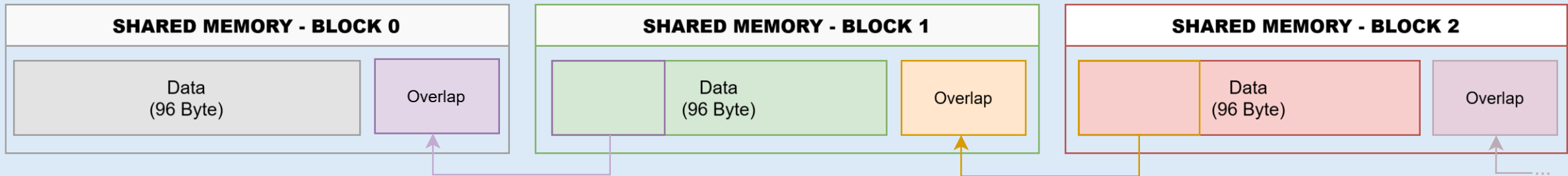
STRIDE CODE: Fetching Phase



1. GLOBAL MEMORY (VRAM)



2. SHARED MEMORY



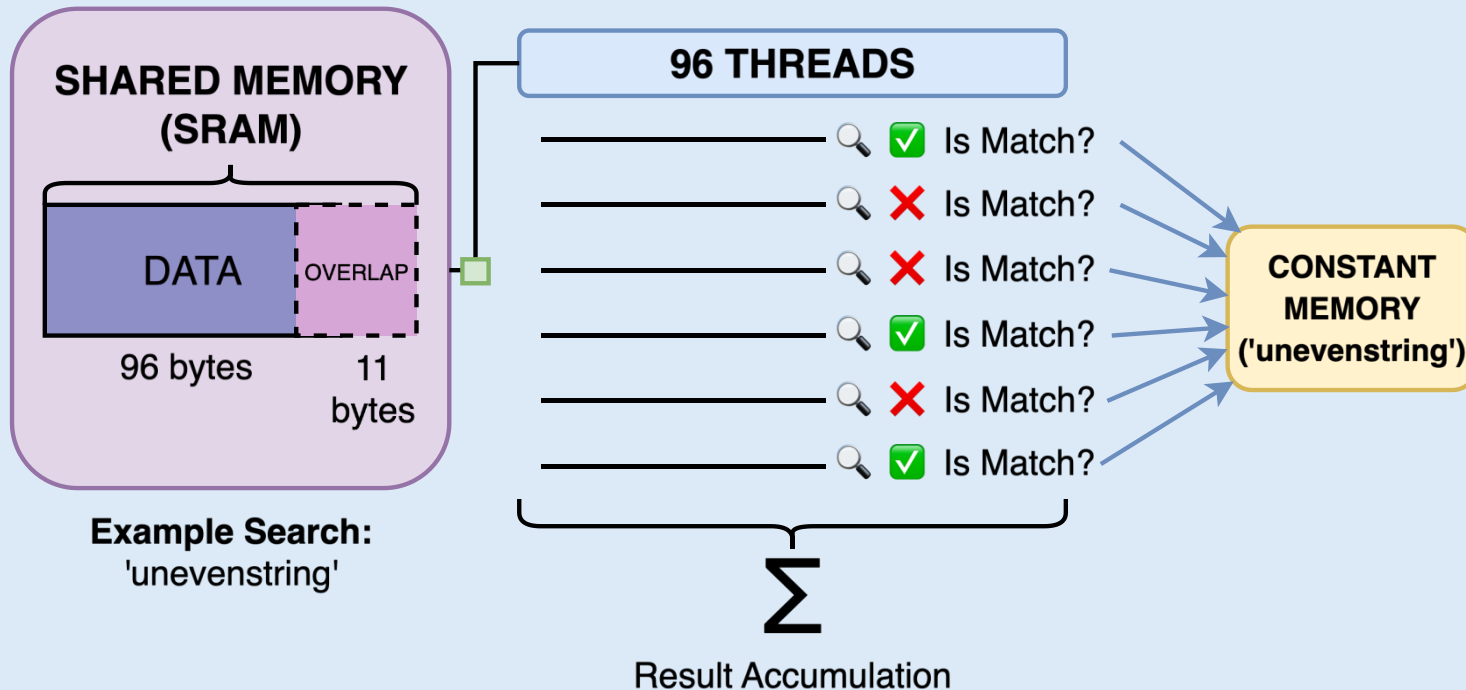
Number of threads per block:
96

Required bytes per block:
 $\text{Number of threads per block} + \text{string_length} - 1$

STRIDE CODE: Comparison Phase



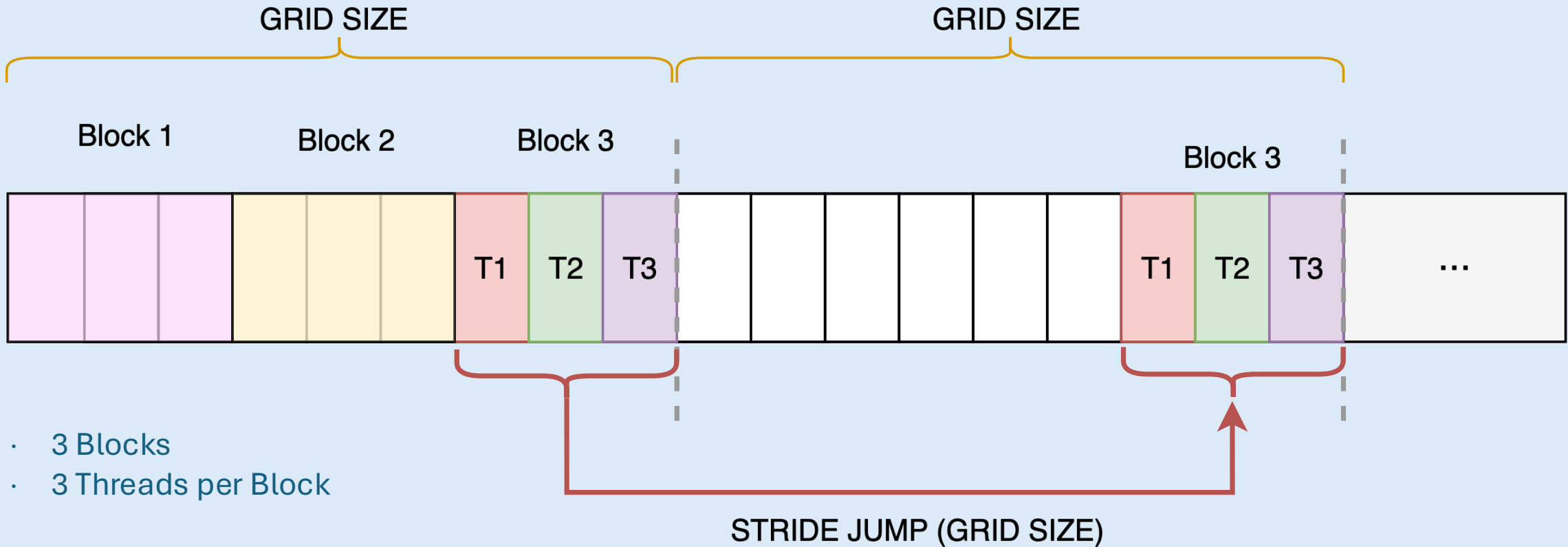
MATCH PHASE



Comparison and Matching

- Each thread independently analyzes the text starting from adjacent indexes within the SRAM
- Aborts the string comparison at the first mismatched character
- Matches are recorded by each thread in a local counter.

STRIDE CODE: Stride Jump



The block moves to the next position only after all threads have reached the synchronization barrier

STRIDE CODE

CudaOccupancyMaxActiveBlocksPerMultiprocessor()

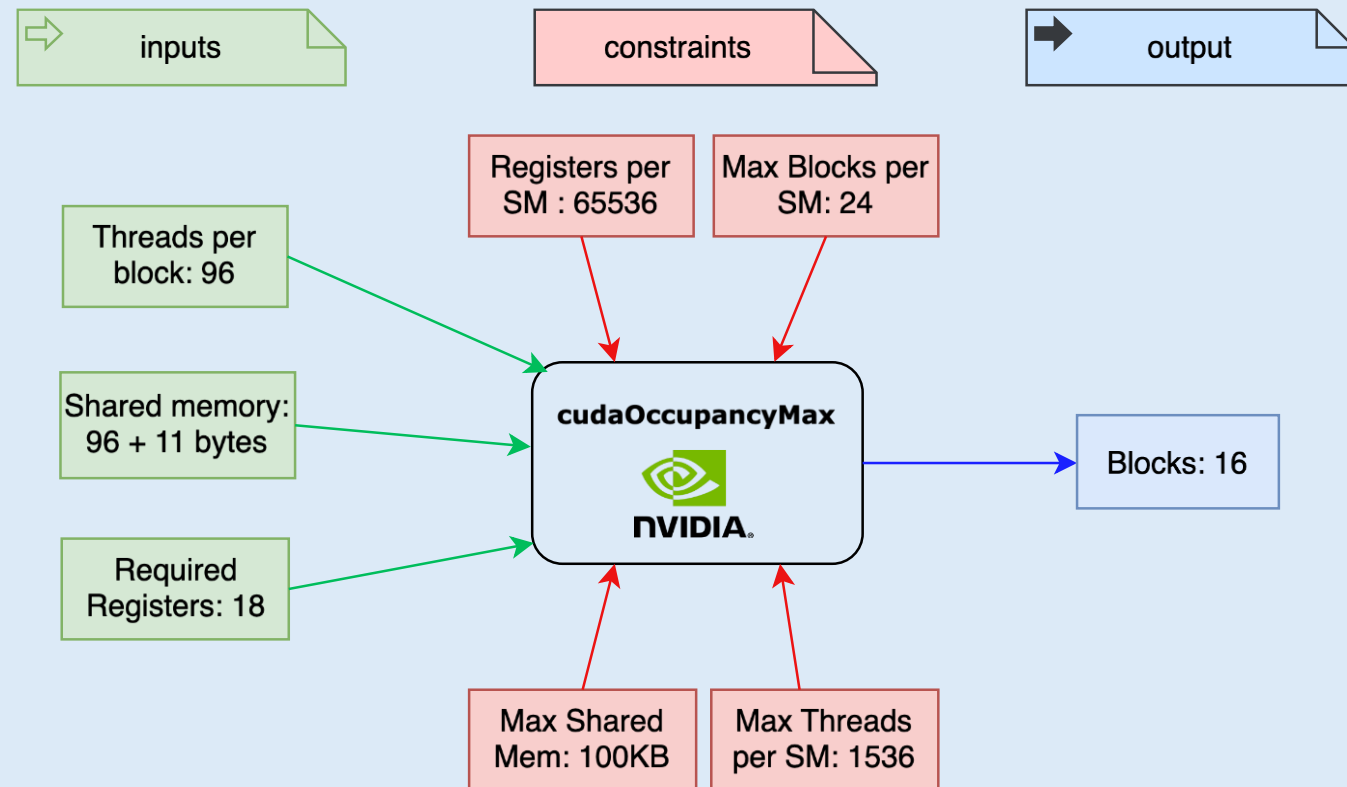


How it works:

Determines the best block configuration to maximize occupancy, based on:

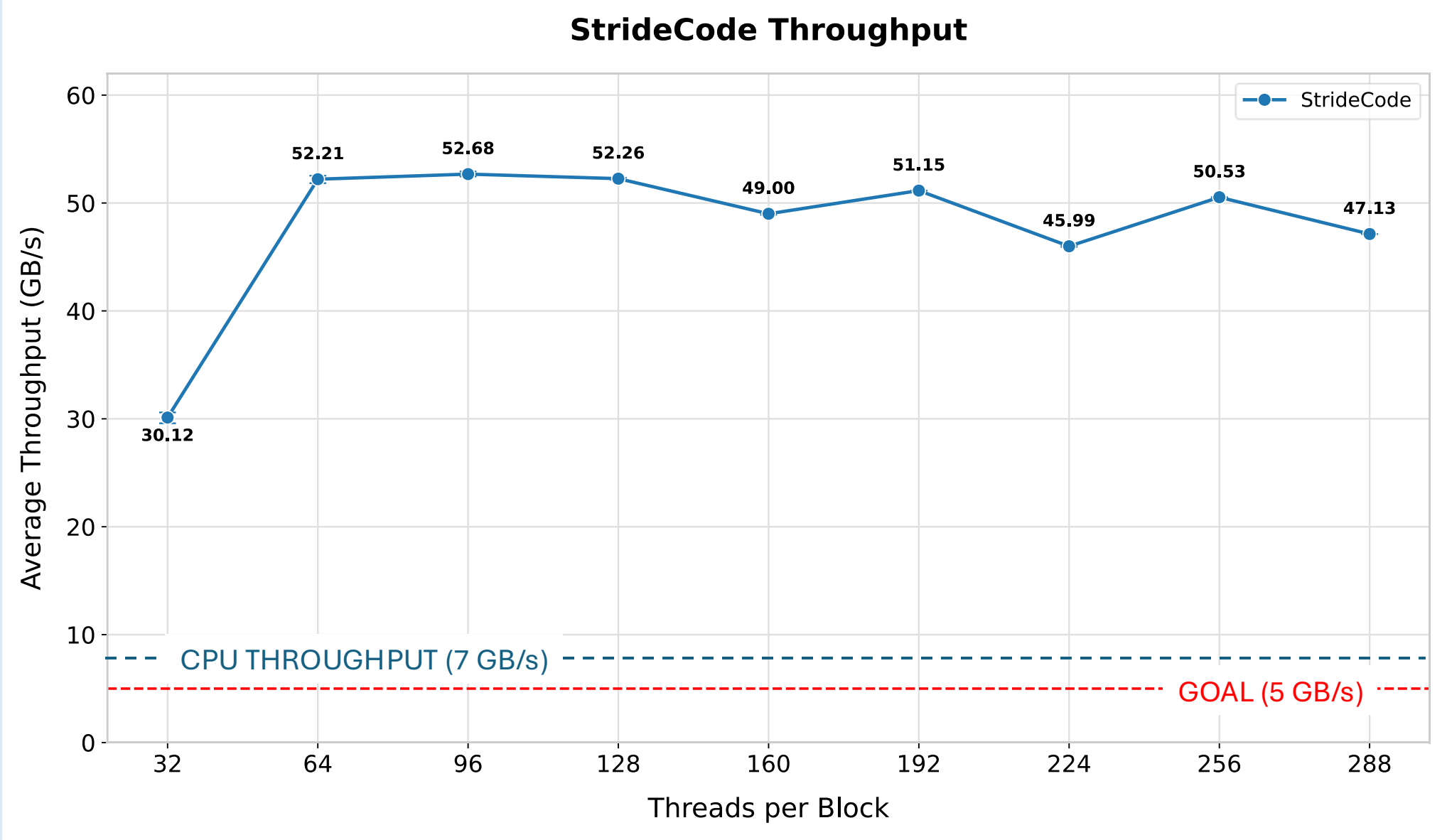
1. Available vs required physical registers by the kernel
2. Available vs requested shared memory per block
3. The architectural limit of threads supported by the SM.
4. Number of threads per block

- **CUDA built in function**



Example with: “unevenstring” (length: 12 bytes)

STRIDE CODE: Throughput Analysis



STRIDE CODE: Problems



Problem - The 1.33GB Fetch Overhead

Issue: Performance Penalty

- The profiler shows that 5.33GB of memory loads are required to process a 4GB file

$$\text{Additional load} = \frac{1.3GB}{5.3GB} = 25\%$$

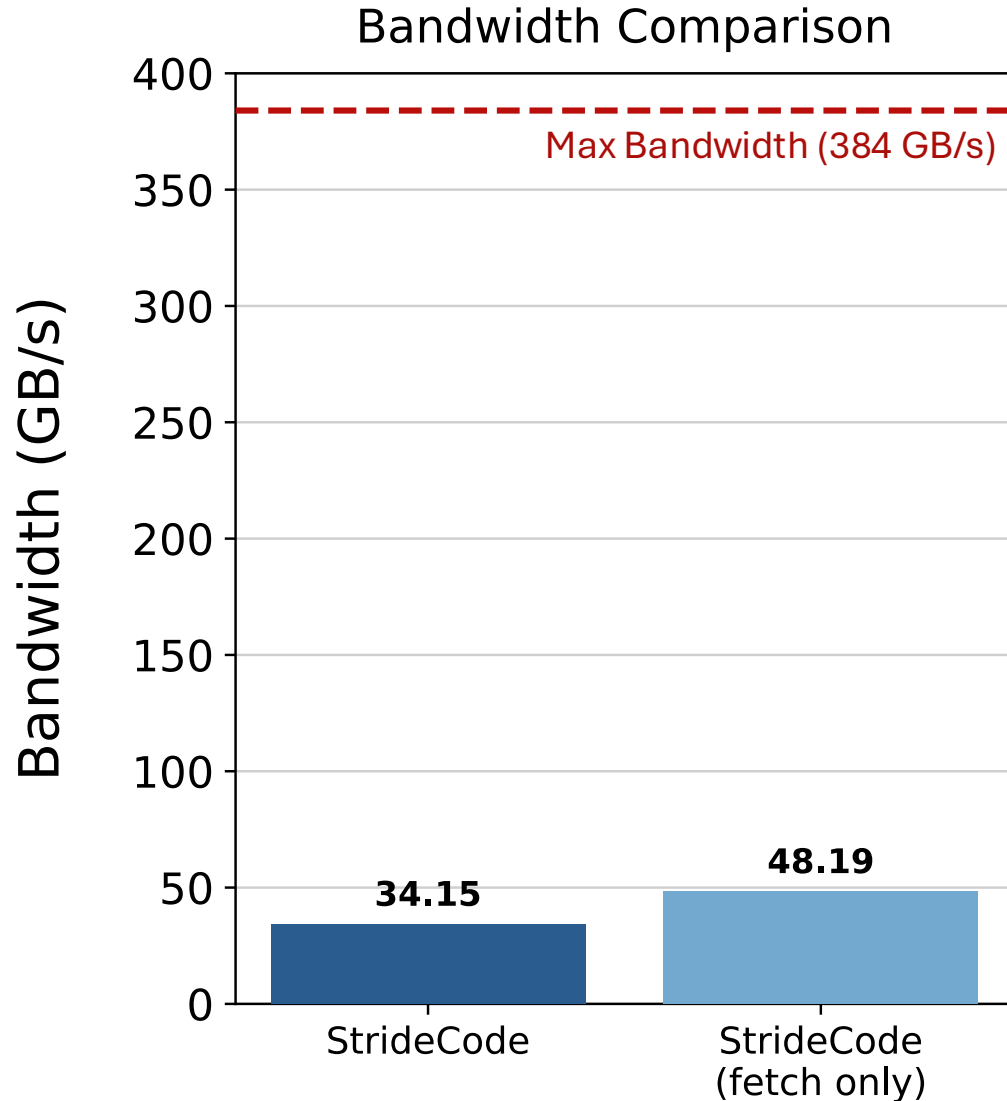
Cause: Hardware Granularity

- Transactions are performed in 32-byte segments.
- Fetching the overlap forces the hardware to read 32 bytes.

Consequence

- Processing 4GB in 96-byte steps triggers 43.690.667 fetch cycles.
- For each fetch cycle the overlap is rounded up to 32
- Fetch cycles multiplied by the 32-byte segment generates 1.33GB of repeated reads.

STRIDE CODE: Problems



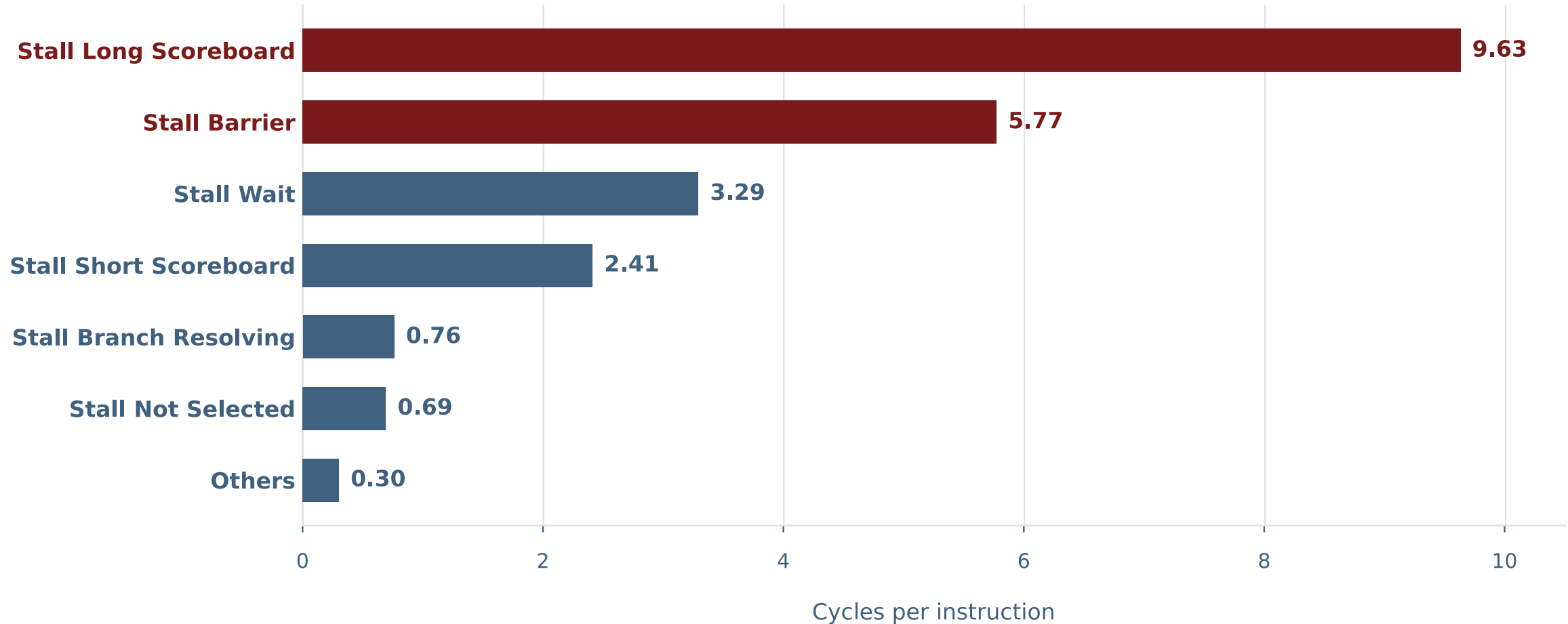
Problem - Memory Bus Under-Utilized

- Caused by 32-byte hardware granularity, this redundant traffic is almost entirely absorbed by the L2 Cache.
- The memory controller is flooded with billions of granular, character-by-character load instructions.
- The physical bus achieves only ~48 GB/s (out of 384 GB/s), even with the string matching logic disabled.

STRIDE CODE: Problems



Problem – Stall Long Scoreboard and Stall Barrier



STRIDE CODE: Problems

Problem – Stall Long Scoreboard and Stall Barrier

Stall Long Scoreboard

- Profiling reveals that the majority of Scoreboard stalls are due to a single instruction (LDG.E.U8)
- The instruction is responsible for global memory fetches

Instruction	Attributed Stalls
LDG.E.U8 R6, desc[UR12][R6.64]	72%

Stall Barrier

- Processing a 4GB file in 96-byte steps requires 43.690.667 loop iterations.
- With 2 sync barriers per iteration, each block halts over 210.000 times.

STRIDE CODE: Problems



Problem 1:

**Stall Long
Scoreboard**

Problem 2:

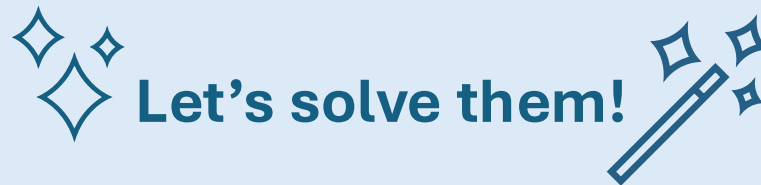
**GPU Bus
Under-Usage**

Problem 3:

**Memory Fetch
Overhead**

Problem 4:

Stall barrier



Let's solve them!



NEW IMPLEMENTATION



STRIDE CODE



STRIDE TAIL

STRIDE TAIL: Solutions



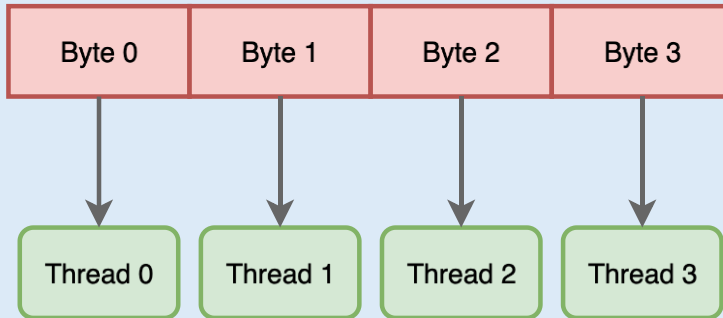
Solution - GPU Memory Bus Under-Utilization

Research literature ([source](#)) states:

"A simple way to improve the bandwidth between the device memory and an SM, is to have each thread input 16 characters at a time, process these 16 characters, and write the output values for these 16 characters to device memory."

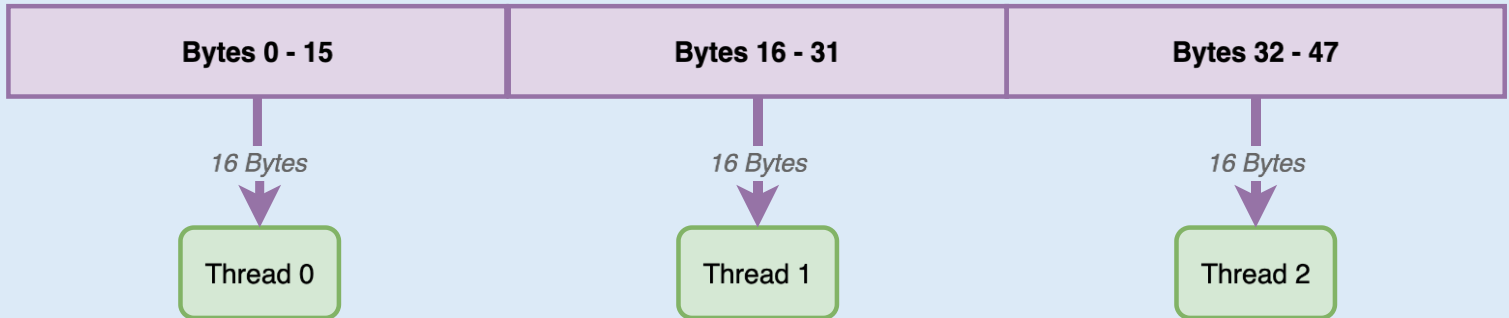
1. Standard Transfer (1 Byte per Thread)

VRAM

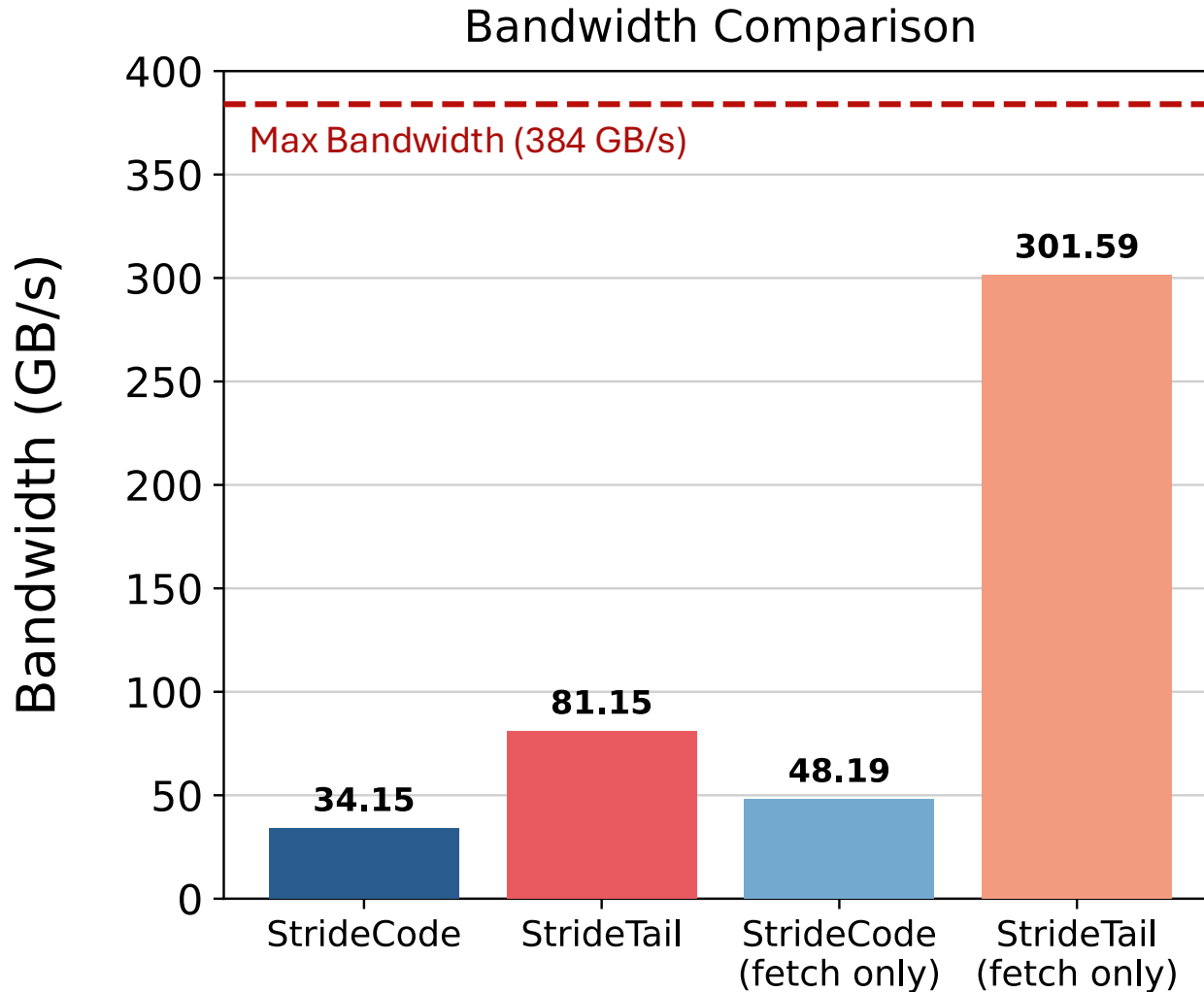


2. Vectorial Transfer (16 Byte per Thread)

VRAM



STRIDE TAIL: Solutions



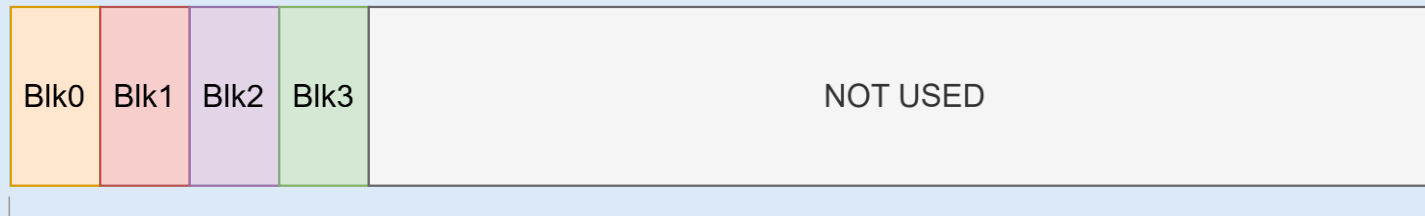
Usage of vector instructions

- **Workflow:** Instead of byte-by-byte reads, we use 16-byte aligned parallel fetches
- **Benefit:** Fetch instructions drop by a factor of 16
- **Metric:** *Stall Long Scoreboard* drops from 9.63 to 1.23 cycles per instruction.

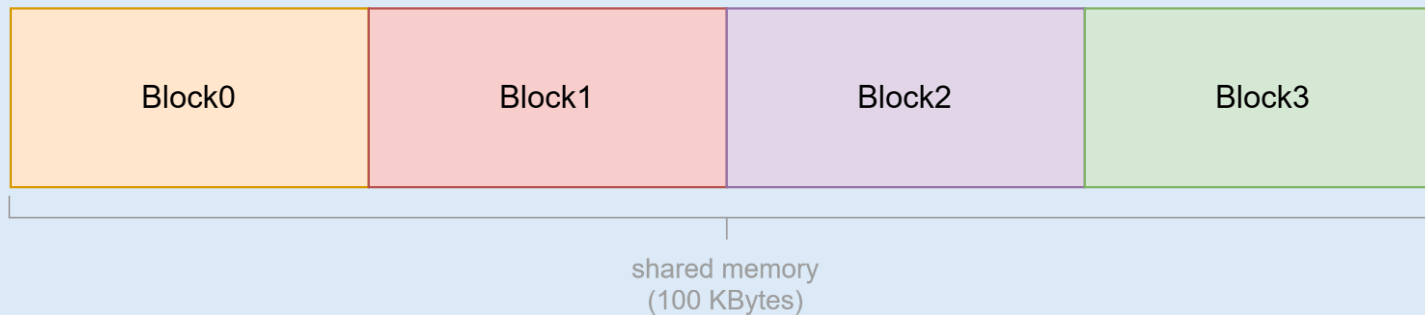
STRIDE TAIL: Solutions



BEFORE (STRIDE CODE)



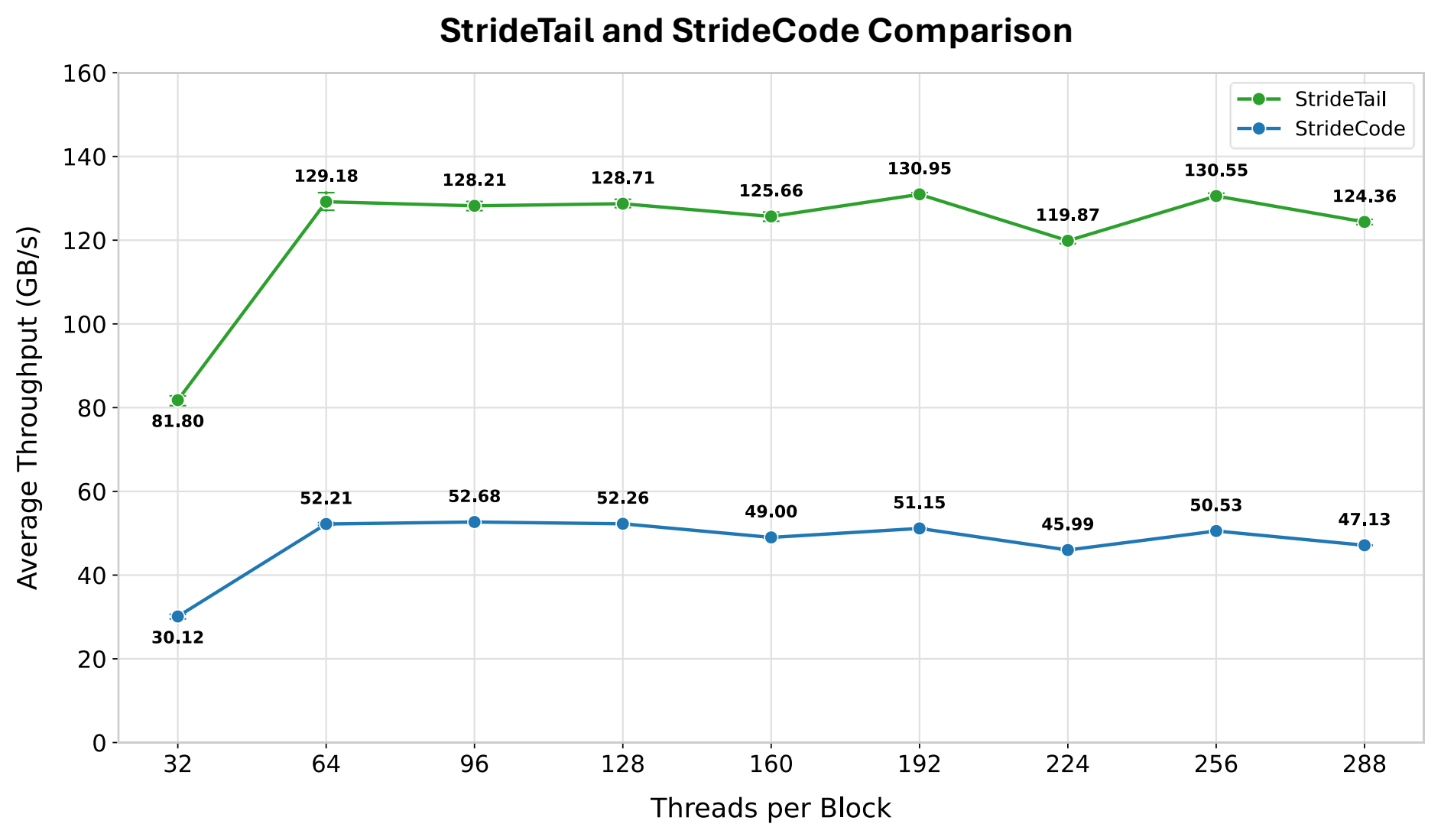
AFTER (STRIDE TAIL)



Usage of shared memory

- **Lower Fetch Overhead:** Processing 5-10KB chunks per block greatly reduces overlap duplication. The fetch overhead is now **~20 MB** (from 1,33 GB)
- **Crushing the Stall Barrier:** Larger data chunks require drastically fewer *syncthread()* calls (from 210.000 to ~2.000)
- **Metric:** *Stall Barrier* plummets from **5.77** to **0.7** cycles per instruction.

STRIDE TAIL: Throughput Analysis



STRIDE TAIL: Tail Effect



Looking at the profiler...

NVIDIA Nsight Compute - Performance Advisor



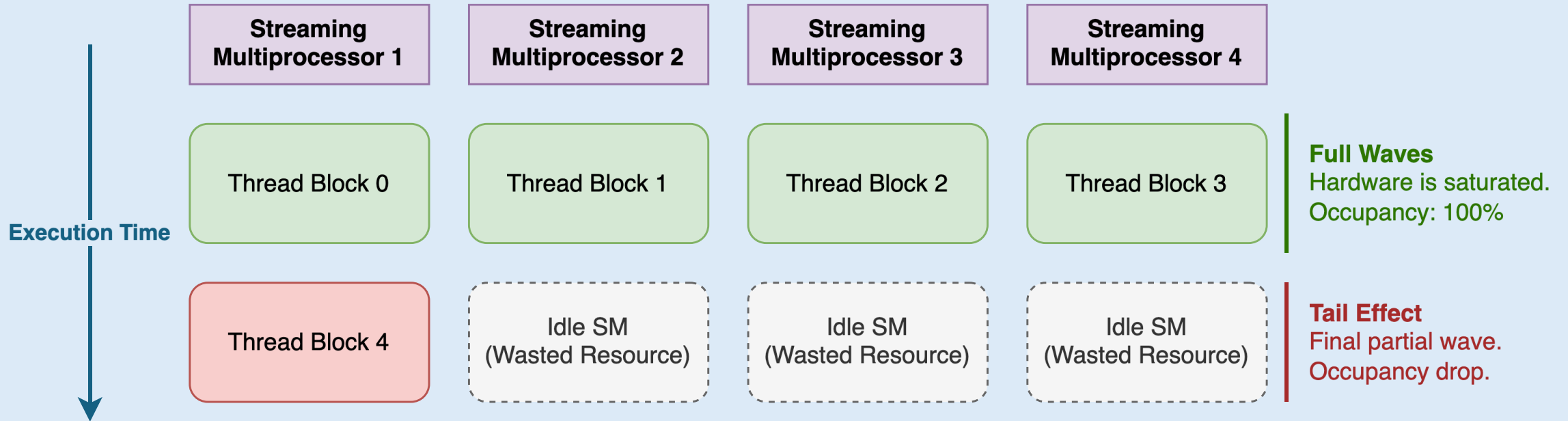
Tail Effect

Est. Speedup: 50.00%

A wave of thread blocks is defined as the maximum number of blocks that can be executed in parallel on the target GPU. The number of blocks in a wave depends on the number of multiprocessors and the theoretical occupancy of the kernel. This kernel launch results in 1 full waves and a partial wave of 26 thread blocks. Under the assumption of a uniform execution duration of all thread blocks, this partial wave may account for up to 50.0% of the total runtime of this kernel. Try launching a grid with no partial wave. The overall impact of this tail effect also lessens with the number of full waves executed for a grid. See the [Hardware Model](#) description for more details on launch configurations.

► Key Performance Indicators

STRIDE TAIL: Tail Effect

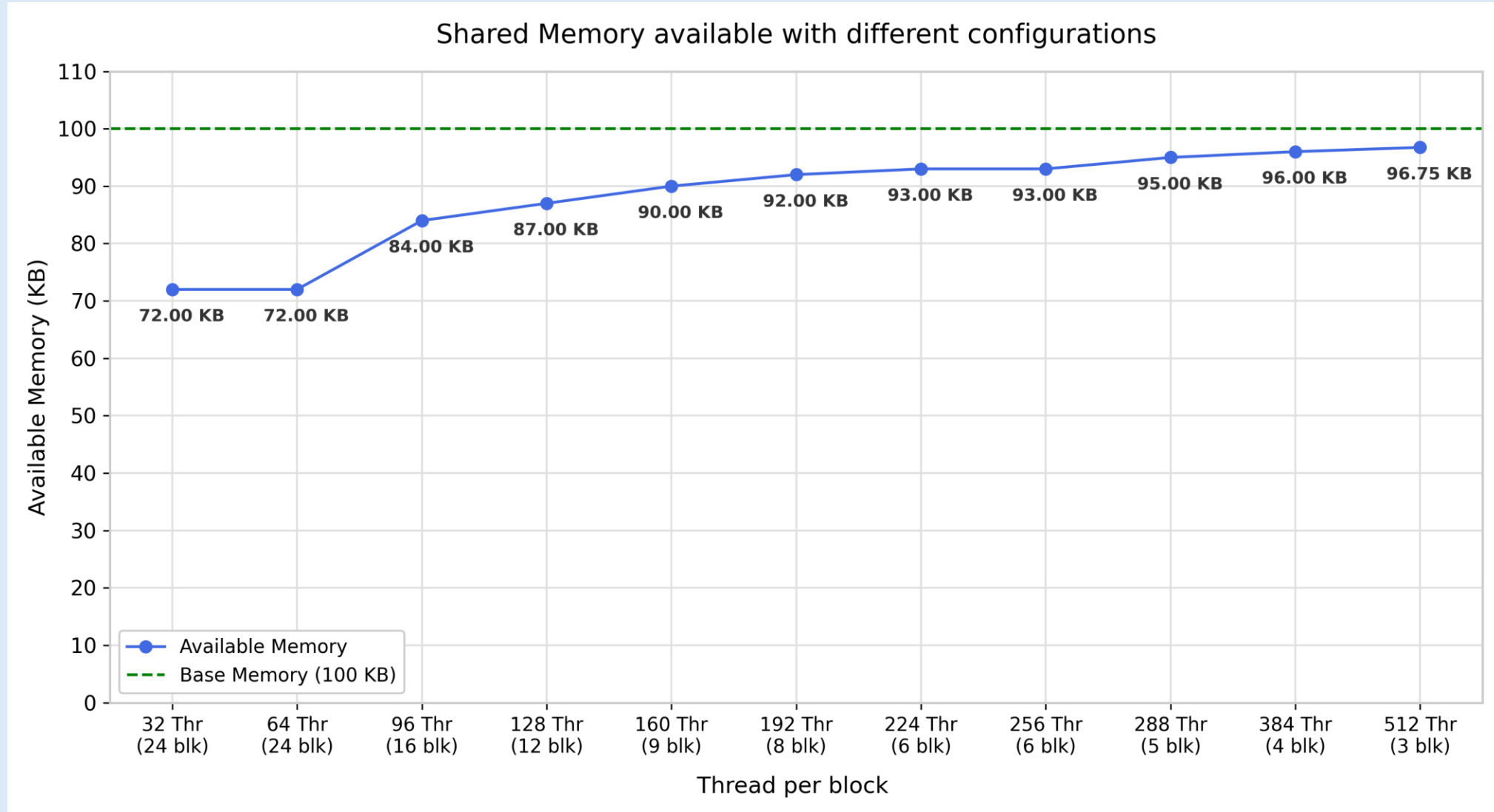


Executing **Stride Tail** with **192 threads per block** yields the following results:

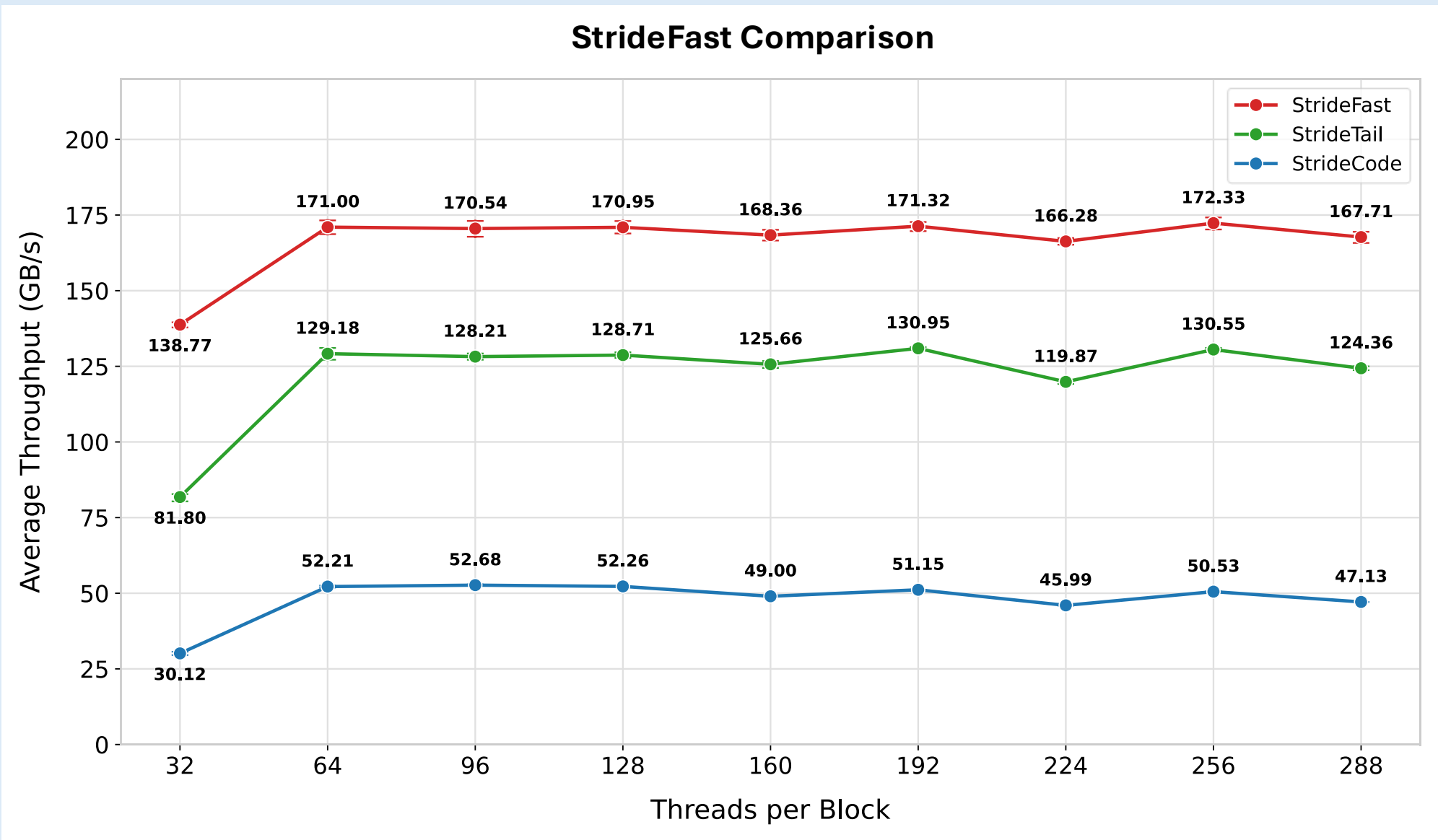
- **Theoretical threads blocks per SM** = $\frac{\text{total number of threads}}{\text{threads per block}} = \frac{1536}{192} = 8$
- **Active threads blocks per SM** = 7
- **Grid size** = theoretical threads blocks x SM = 8 x 26 = 208
- **Blocks per wave** = active threads blocks x SM = 7 x 26 = 182

TAIL
208-182 = 26

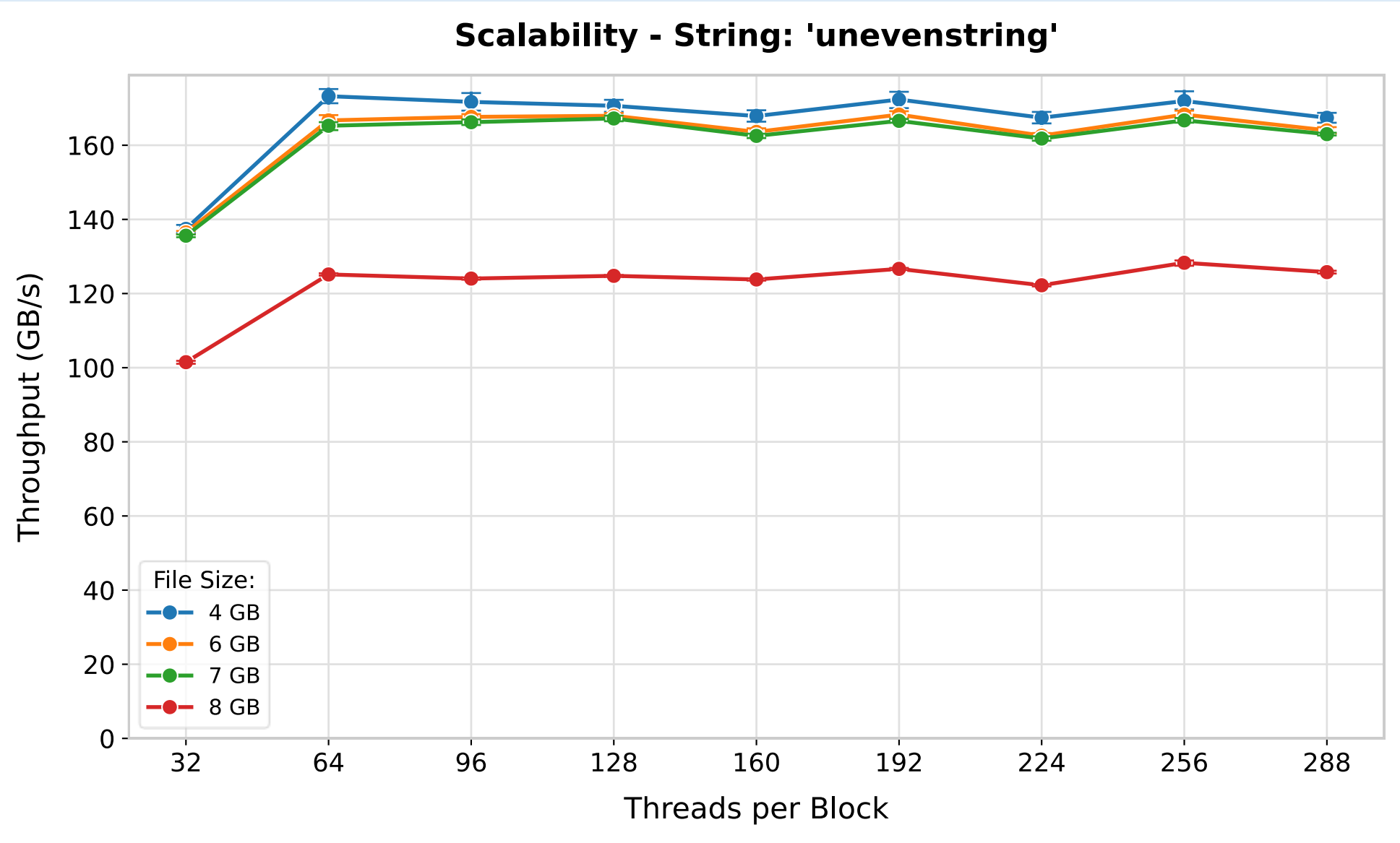
STRIDE TAIL: Tail Effect



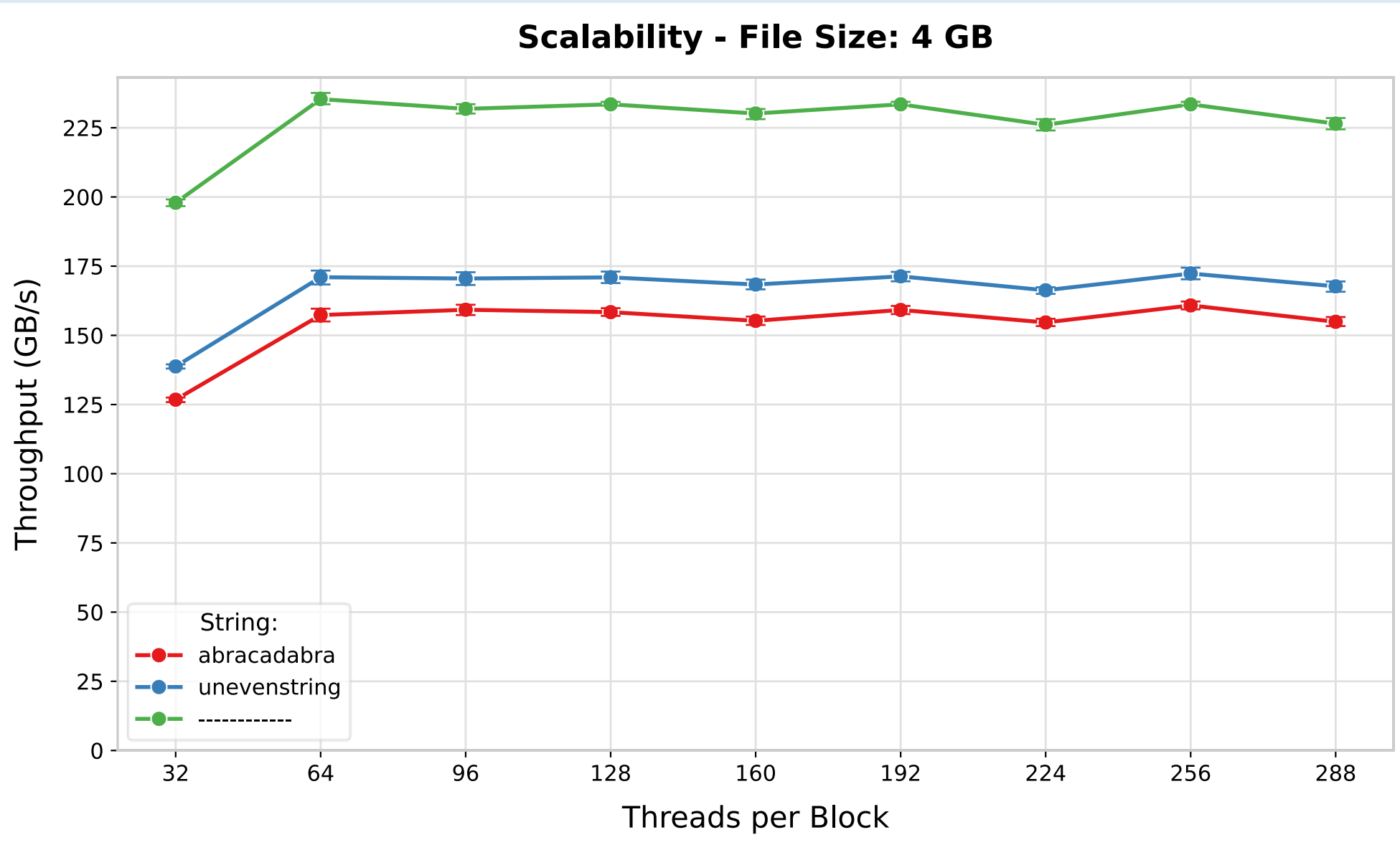
STRIDE FAST: Throughput Analysis



Scalability: different file size



Scalability: different string distribution



**Master Degree Computer Engineering
Computer Architecture**

Thanks for the attention

**Megan Maremmani
Eleonora Sgorbini
Sebastiano Pala**